

Rapport de Devoir : Systèmes Multi-Agents

Atelier de Production Distribué

Tom DAVID & Titouan PASQUIER

Janvier 2026

1 Introduction

Ce projet implémente une simulation d'atelier de production basée sur une architecture multi-agents utilisant la plateforme JADE. L'objectif est de résoudre un problème d'ordonnancement décentralisé où des produits nécessitant plusieurs compétences circulent entre des robots hétérogènes. La solution repose sur une variante du protocole *Contract Net* pour optimiser le temps de traitement global (Makespan) de manière dynamique.

2 Choix de Conception et Représentation

2.1 Représentation des Produits

Pour permettre la circulation des objets entre les agents via le réseau, la classe `Produit` implémente l'interface `java.io.Serializable`. Chaque produit est défini par :

- Un identifiant unique.
- Une `HashMap<Integer, Boolean>` représentant les compétences requises. La clé est l'ID de la compétence, la valeur indique si elle est réalisée (`true`) ou non (`false`).
- Un compteur d'échecs (`nbEchecs`) permettant à l'Atelier de calculer des statistiques sur la fiabilité de la chaîne.

2.2 Architecture des Agents

Tous les agents héritent d'une classe abstraite commune `BaseAgent`, qui centralise :

- Le chargement de la configuration via un fichier `.properties`.
- L'enregistrement auprès du Directory Facilitator (DF).
- La collecte de statistiques (surcharge de la méthode d'envoi pour compter les messages).

3 Protocole de Communication et Stratégie

Nous avons implémenté une version adaptée du **Contract Net Protocol** (CNP). Contrairement à une approche gloutonne où un robot garderait un produit s'il possède la compétence, notre système privilégie l'optimum global.

3.1 Logique d'Enchère Systématique

Lorsqu'un robot reçoit un produit (de l'Atelier ou d'un autre robot), il lance systématiquement une enchère pour la prochaine compétence requise :

1. Le robot émetteur envoie un CFP (Call For Proposal) à tous les agents enregistrés comme "robots".
2. Chaque robot (y compris l'émetteur s'il a la compétence) calcule son *Makespan* estimé.
3. Le robot émetteur compare les offres reçues (PROPOSE) et sa propre estimation.
4. Le produit est attribué (ACCEPT_PROPOSAL) à l'agent minimisant le temps total.

3.2 Calcul du Makespan

Le critère de décision est le temps espéré de fin de traitement. Il prend en compte :

- La longueur de la file d'attente actuelle du robot.
- Le temps de traitement théorique (λ_3).
- La probabilité d'échec (loi géométrique), pénalisant les robots peu fiables.

4 Implémentation JADE et Comportements

Les agents utilisent différents types de **Behaviours** pour assurer asynchronisme et réactivité.

4.1 Agent Atelier

- **TickerBehaviour** : Génère périodiquement de nouveaux produits selon une loi uniforme entre λ_1 et λ_2 .
- **CyclicBehaviour** : Écoute en permanence les messages de retour (INFORM pour succès, FAILURE pour échec définitif) pour mettre à jour les statistiques globales.

4.2 Agent Robot

- **CyclicBehaviour (Mailbox)** : Ce behavior est lancé à la création des robots. Routeur de messages. Il analyse le performatif (CFP, REQUEST, ACCEPT_PROPOSAL) et délègue le traitement. Pour cela il utilise un MessageTemplate avec plusieurs OR imbriqué.
- **TickerBehaviour (Worker)** : Ce behavior est lancé à la création des robots. Simule le travail. À chaque "tick", il vérifie si une tâche est terminée et détermine aléatoirement le succès ou l'échec en fonction de la compétence.
- **Behaviour (Machine à états)** : Est instancié quand un robot doit faire une enchère. Utilisé pour la gestion d'une enchère (EnchereBehaviour). Ce comportement gère séquentiellement l'envoi du CFP, l'attente des réponses (avec timeout) et l'attribution.

4.3 Performatifs ACL Utilisés

Performatif	Usage dans le système
REQUEST	Injection initiale d'un produit par l'Atelier vers un Robot.
CFP	Lancement d'une enchère pour une compétence spécifique.
PROPOSE	Réponse d'un robot indiquant son Makespan estimé.
REFUSE	Réponse d'un robot ne possédant pas la compétence.
ACCEPT_PROPOSAL	Notification au vainqueur de l'enchère (transfert du produit).
INFORM	Retour du produit fini à l'Atelier.
FAILURE	Signalement d'un échec critique (aucun robot disponible).

5 Script de Lancement et Configuration

Le projet est fourni avec un script `start_project.sh` qui automatise le déploiement. Ce script :

1. Génère dynamiquement le fichier `config/config.properties` basé sur les variables définies en début de script (nombre de robots, délais λ , compétences).
2. Compile les sources Java.
3. Lance le conteneur principal JADE avec l'agent Atelier et le nombre spécifié d'agents Robots.

Cette approche permet de tester rapidement différentes configurations (ex : augmenter la charge avec λ_1 faible ou augmenter le nombre de robots) sans modifier le code source.